DeployGuard

Panel Answer Guide - 140 Model Answers

Final Year Project II - Implementation

Project	DeployGuard - Deterministic PaaS Deployment Control Plane
Group Members	Hasan Sajjad Faria Fatima Tania Khawar
Department	Department of Computer Science, Air University Islamabad
Implementation Baseline	GitHub branch test2 - reviewed 13 September 2026

Use these as speaking points, not as text to memorize word-for-word.

Defense rule

Answer from DeployGuard ownership first: what the platform decides, what the execution layer does, what evidence proves success, and what happens on failure. Avoid claiming AWS, Terraform, GitHub or third-party components as your original invention.

Project, Problem and Contribution

1. What is DeployGuard in one sentence?

Model answer: DeployGuard is a PaaS-style deployment control plane that converts a supported GitHub repository into a verified, observable and reversible cloud release.

2. What real problem does DeployGuard solve?

Model answer: It removes the coordination burden across repository analysis, container build, CI/CD, cloud infrastructure, runtime verification, security, state, cost and recovery.

3. What is the main contribution of your FYP?

Model answer: The contribution is the deterministic control plane and lifecycle model, not the underlying cloud tools: immutable contracts, release generations, evidence-driven verification, rollback/destroy correctness and operator UX.

4. Why is this more than a CI/CD pipeline?

Model answer: A CI/CD pipeline executes steps; DeployGuard also owns repository interpretation, configuration admission, cloud lifecycle, release truth, rollback targets, state reconciliation, monitoring, cost and troubleshooting.

5. Why did you choose a PaaS problem?

Model answer: Deployment is a high-friction systems problem where cloud, DevOps, security and developer experience intersect, making it suitable for an implementation-heavy FYP.

6. Who is the target user?

Model answer: Developers or small teams that want self-service deployment without manually coordinating every AWS and DevOps primitive.

7. What is the FYP-II focus compared with FYP-I?

Model answer: FYP-II is implementation and proof: integrated system behavior, executable contracts, cloud deployment lifecycle, monitoring, recovery and demonstration.

8. What is currently in scope?

Model answer: Supported JavaScript and Python web applications, GitHub source, containerized AWS runtime, managed databases, security, observability, cost and lifecycle operations.

9. What is out of scope?

Model answer: Universal support for every language/runtime, pure static repositories without an application runtime, and hyperscale multi-cloud production economics.

10. What makes the system academically defensible?

Model answer: It has explicit research motivation, clear ownership boundaries, reproducible contracts, measurable lifecycle behavior and a verification strategy rather than only a UI demo.

Architecture and Control Plane

11. Explain the architecture at a high level.

Model answer: Developer UI -> control plane -> workflow/build execution -> image registry -> Terraform/AWS runtime -> verification -> stable release and operational evidence.

12. What is the control plane?

Model answer: The backend logic that owns deployment intent, source/build/runtime contracts, lifecycle state, failure classification and release promotion.

13. What is the data plane/runtime?

Model answer: The cloud resources that actually run application and database containers and serve traffic.

14. Why separate control plane from runtime?

Model answer: It isolates product decisions and state from low-level execution, making lifecycle behavior more deterministic and testable.

15. What is the source of truth for infrastructure?

Model answer: Terraform state and declarative infrastructure definitions are the infrastructure source of truth.

16. What is the source of truth for the active release?

Model answer: DeployGuard's stable release and service revision records, validated against runtime evidence.

17. Why have multiple authorities at all?

Model answer: Different layers own different concerns: Terraform owns resource existence, DeployGuard owns release acceptance, and ECS owns scheduling/deployment primitives.

18. How do you prevent Terraform and ECS from fighting?

Model answer: The ownership boundary preserves platform-managed runtime changes such as active task revision while keeping Terraform authoritative for resource existence and destroy.

19. What happens when the platform restarts?

Model answer: Persistent project, operation, revision, release and state records allow it to reconstruct current state and reconcile with external systems.

20. Why is state management important in PaaS?

Model answer: A deployment spans asynchronous external systems; stale or contradictory state can create duplicate releases, broken rollback or orphaned resources.

21. What does deterministic mean in your project?

Model answer: The same accepted source/configuration identity resolves to the same normalized deployment facts and can be traced through the release.

22. What is a canonical contract?

Model answer: A normalized versioned representation of an important deployment decision, such as build target or runtime configuration.

23. Why not recalculate everything at each stage?

Model answer: Repeated detection can drift as files, defaults or state change. Persisted canonical contracts give every stage the same authoritative input.

24. How is immutability used?

Model answer: Exact Git SHA, fingerprints, runtime revisions, image digests and release identities prevent ambiguous “latest” behavior.

25. What is the benefit of fingerprints?

Model answer: They allow the platform to detect whether logically important configuration changed and to prove which contract produced a release.

26. How is multi-service architecture handled?

Model answer: Each deployable service has its own identity, directory, port and runtime revision, while the project coordinates them under one lifecycle.

27. How do you avoid one service contaminating another?

Model answer: Service boundaries, environment ownership, directory normalization and per-service runtime identity are enforced before execution.

28. Why fail closed on ambiguity?

Model answer: Deploying the wrong service or port is worse than explicitly reporting unsupported/ambiguous input.

29. What is the role of queues?

Model answer: BullMQ/Redis decouple long-running lifecycle work from HTTP requests and support controlled asynchronous execution.

30. What database stores control-plane state?

Model answer: PostgreSQL through TypeORM stores projects, pipeline runs, revisions, releases, audit, billing and operational records.

Repository Analysis and Build

31. How does DeployGuard know what to build?

Model answer: It resolves a canonical build target from the exact source tree and selected service rather than relying on a single framework guess.

32. What fields are important in the build target?

Model answer: Source SHA, service/build/workspace/install roots, package identity, deployment contract, execution facts, dependency paths, evidence and fingerprint.

33. Which deployment contracts are currently supported?

Model answer: JavaScript standalone, JavaScript workspace member and Python standalone.

34. How are JavaScript monorepos handled?

Model answer: Workspace ownership and package identity are resolved from manifests/workspace declarations and conflicting package-manager evidence is rejected.

35. Which JavaScript package managers are recognized?

Model answer: npm, pnpm, yarn and bun within the supported contract.

36. How are Python services handled?

Model answer: Standalone Python evidence is recognized from package/configuration files; sibling dependencies outside the service boundary are rejected when unsafe.

37. How does the build subsystem work?

Model answer: A build worker receives source plus the canonical build specification, materializes a deterministic container build, validates the image and publishes it.

38. Why keep the build engine behind an abstraction?

Model answer: The academic contribution is deterministic build orchestration and contract ownership; the low-level container builder is an implementation dependency.

39. Does DeployGuard require a user Dockerfile?

Model answer: No. The normal path builds from repository evidence and the platform's normalized build specification.

40. Do you support a fallback build path?

Model answer: A narrow certified fallback exists only for proven internal build-engine failure under a supported template contract; it is not a generic recovery for application errors.

41. How do you resolve the service port?

Model answer: Evidence is evaluated by priority: explicit config, source binding, start-command flags, environment and framework defaults.

42. What happens when two different ports are detected at the same priority?

Model answer: The resolver reports a conflict and blocks the deployment rather than guessing.

43. What happens when no port can be resolved?

Model answer: The service fails admission with a deterministic unresolved-port error.

44. Which common framework defaults are known?

Model answer: Examples include Next/Nest/Nuxt 3000, Vite 5173, Django/FastAPI 8000 and Flask 5000, with explicit evidence taking precedence.

45. Why does the platform own PORT and HOST?

Model answer: They are runtime transport values required for consistent container reachability and load-balancer integration.

46. What prevents repository paths from escaping the repo?

Model answer: Canonical path normalization and real-path containment checks reject invalid service directories.

47. Why pin the exact Git commit SHA?

Model answer: A branch name can move while deployment is running; the SHA makes source identity immutable.

48. What is an immutable image?

Model answer: A registry artifact identified by immutable release identity/digest rather than an ambiguous mutable tag.

49. Why validate an image before deployment?

Model answer: It catches build/runtime packaging defects before provisioning or promoting cloud resources.

50. Can DeployGuard deploy a pure HTML/CSS site?

Model answer: Not as a raw static-file repository in the current contract; it can deploy framework/application-served static content.

GitHub and CI/CD

51. Why use GitHub Actions?

Model answer: It provides an auditable execution environment close to the source repository and integrates with OIDC, workflows and release evidence.

52. How does DeployGuard trigger deployment?

Model answer: The control plane seals inputs and dispatches a reusable workflow with immutable project/source/runtime identifiers.

53. Why use OIDC instead of AWS keys in GitHub?

Model answer: OIDC provides short-lived federated credentials and avoids distributing long-lived cloud keys to repositories.

54. What does the AWS trust policy verify?

Model answer: The GitHub OIDC provider, STS audience and an authorized repository subject pattern.

55. What if the trust policy is wrong?

Model answer: The platform treats it as a platform configuration boundary and fails before unsafe cloud mutation.

56. How do you know the workflow has enough AWS permissions?

Model answer: DeployGuard has an explicit lifecycle-specific AWS capability contract and verifies required capabilities before dispatch.

57. Which AWS areas are covered by the capability model?

Model answer: STS, ECR, ECS, EC2, load balancing, logs, IAM, secrets, EFS, S3, service discovery and Route 53.

58. Why make permissions lifecycle-specific?

Model answer: Deploy, rollback and destroy need different actions; modeling paths separately reduces ambiguity and improves diagnostics.

59. What if the workflow never starts?

Model answer: Dispatch state is persisted and classified separately from a workflow execution failure.

60. How is workflow evidence correlated back to DeployGuard?

Model answer: Operation ID, project ID, source SHA, service/revision identity and result artifacts connect external execution to control-plane state.

61. What happens if a GitHub step fails?

Model answer: The terminal failure path captures bounded evidence, stage and ownership metadata for UI, recovery and troubleshooting.

62. Do you depend on branch names after dispatch?

Model answer: No. The immutable commit SHA remains the release source identity.

63. How are concurrent operations controlled?

Model answer: Operation state/leases and lifecycle admission prevent conflicting active deployments for the same project scope.

64. Why is external action pinning relevant?

Model answer: Pinning third-party actions to immutable versions reduces supply-chain drift in CI/CD.

65. What is the difference between workflow execution and release truth?

Model answer: A successful workflow is evidence; DeployGuard still validates runtime/release contracts before marking a release stable.

AWS and Infrastructure as Code

66. Why AWS ECS Fargate?

Model answer: It runs containers without managing EC2 hosts and integrates naturally with ECR, ALB, IAM, CloudWatch and task definitions.

67. Why not Kubernetes for the FYP?

Model answer: ECS reduces orchestration complexity while still demonstrating container scheduling, service deployment, health and cloud lifecycle. Kubernetes could be a future provider.

68. Why use Terraform?

Model answer: It provides declarative, versioned infrastructure, remote state, plan/apply semantics and deterministic destroy.

69. What resources does Terraform create?

Model answer: Project ECS resources, load balancing, security groups, IAM execution role, logging, secrets, persistent database storage and service discovery, plus state-backed lifecycle resources.

70. Where is Terraform state stored?

Model answer: In an S3 remote backend scoped to the project/runtime state path.

71. Why is remote state important?

Model answer: It persists infrastructure identity outside the local process and supports consistent later plan, rollback and destroy operations.

72. What is configuration drift?

Model answer: A difference between intended/declarative infrastructure and actual cloud state.

73. How does DeployGuard reduce drift?

Model answer: Infrastructure mutations are centralized through Terraform and reconciliation logic rather than manual cloud-console setup.

74. What is the ALB doing?

Model answer: It provides a public HTTP entry point and routes to the service target group while exposing target-health evidence.

75. Why use security groups per service?

Model answer: They establish explicit network boundaries between public load balancer, application and managed database traffic.

76. How does the runtime know if an app is reachable?

Model answer: A platform transport probe and target-health checks verify that the application port is accepting connections.

77. Why have a platform health path separate from the app path?

Model answer: Transport readiness can be measured consistently even when application-specific health endpoints differ.

78. How are logs collected?

Model answer: Application and database tasks use CloudWatch log groups correlated to project/service identities.

79. How are secrets delivered to ECS?

Model answer: Task definitions reference AWS Secrets Manager values; sensitive data is kept separate from normal environment values.

80. Why use an ECS execution role?

Model answer: It allows the task runtime to pull images, write logs and retrieve authorized secret values.

81. What is Cloud Map used for?

Model answer: Private DNS service discovery for the managed database endpoint inside the VPC.

82. Why is EFS used?

Model answer: It provides persistent encrypted filesystem storage for managed database data across task replacement.

83. How do you avoid exposing the database publicly?

Model answer: The database security group allows the database port from the attached application security group rather than from 0.0.0.0/0.

84. What happens if Terraform plan fails?

Model answer: It is classified as a planning failure before apply, preserving a clear infrastructure-stage boundary.

85. What happens if Terraform apply fails?

Model answer: The operation fails as an infrastructure/provider boundary with captured evidence and no stable release promotion.

86. What happens to partially created resources?

Model answer: State and cleanup/reconciliation paths are used to inspect and safely destroy or reconcile residue.

87. Why is destroy still Terraform-owned?

Model answer: The same declarative state that created resources is the safest authority for deleting them consistently.

88. Can later Terraform runs revert an active application release?

Model answer: The ownership model protects platform-managed active release fields so topology reconciliation does not fight the release controller.

89. What is infrastructure namespace/project scoping for?

Model answer: It prevents resource naming and state collisions between projects and supports deterministic cleanup.

90. How would you reduce AWS cost for thousands of apps?

Model answer: Use shared ingress, denser compute, pooled routing, better bin-packing, sleep-to-zero for idle apps and tiered storage/observability instead of dedicated heavy resources per app.

Databases, State and Release Lifecycle

91. Which managed databases are supported?

Model answer: PostgreSQL, MySQL and MongoDB.

92. How are database credentials created?

Model answer: Platform-generated credentials are stored in Secrets Manager and referenced by the runtime.

93. How is database persistence handled?

Model answer: Database data is mounted on encrypted EFS so task replacement does not erase persistent data.

94. How does an application receive database configuration?

Model answer: Canonical aliases provide host, port, user/database values and secret-backed password/URL references to the attached service.

95. What if the application expects a driver-specific database URL scheme?

Model answer: DeployGuard resolves supported URL-scheme evidence and blocks mismatches or conflicting schemes.

96. Why can only one service own the managed database attachment in the current contract?

Model answer: It keeps database ownership and secret injection unambiguous for the current FYP product model.

97. What special case exists for MySQL?

Model answer: The platform verifies both database health and application-user grant readiness before accepting the prerequisite.

98. What is a deployment generation?

Model answer: A persisted candidate/live infrastructure-release generation with explicit lineage and status.

99. Why have generations?

Model answer: They separate candidates from the stable environment and make redeploy, cleanup and rollback history explicit.

100. What states can a generation have?

Model answer: Deploying, live, failed, retired, cleanup pending and cleaned.

101. Can two generations be live at once?

Model answer: The database model enforces a single live generation per project/environment.

102. What is a stable release?

Model answer: The verified release currently accepted as the project's deployment truth.

103. When is a release promoted?

Model answer: Only after cloud/runtime/public verification succeeds.

104. What is redeploy?

Model answer: A new application release on existing project infrastructure, using a new immutable runtime revision and verification.

105. How is rollback implemented?

Model answer: Select an exact previous verified release, activate its immutable runtime revision, wait, verify and then move the stable pointer.

106. Does rollback rebuild the old source?

Model answer: No. It reuses the historical immutable release artifact/revision.

107. Why is no-rebuild rollback safer?

Model answer: It avoids dependency drift and ensures the restored artifact is the same one previously verified.

108. What if rollback verification fails?

Model answer: The stable release is not promoted to the failed rollback target; evidence is recorded for recovery.

109. What does Reset & Deploy Fresh mean?

Model answer: It reconciles/reset lifecycle state using existing safety rules and then creates a fresh deployment identity without silently deleting healthy persistent data.

110. How are stale locks or interrupted operations handled?

Model answer: The state-management layer includes locks, heartbeats, orphan monitoring and recovery/reconciliation services.

Security and DevSecOps

111. What are the main security layers?

Model answer: Federated cloud identity, capability verification, immutable inputs, vulnerability scanning, secret isolation, network security groups, runtime roles, audit and failure sanitization.

112. What is Trivy used for?

Model answer: Container image vulnerability scanning before release, with configurable reporting or enforcement.

113. Can the security scanner block a deployment?

Model answer: Yes, when enforcement is enabled and the image violates platform security policy.

114. Why not scan only source code?

Model answer: Container risk includes OS packages and runtime dependencies; image-level scanning evaluates the artifact that will actually run.

115. How do you prevent secrets appearing in AI prompts?

Model answer: Evidence is sanitized and secret values are represented by controlled references rather than raw configuration wherever possible.

116. How do you prevent secrets in logs?

Model answer: A log sanitizer and bounded evidence model remove or reduce sensitive content before persistence/presentation.

117. What is RBAC used for?

Model answer: To separate ordinary users, privileged roles and admin/operator actions.

118. Why are audit logs important?

Model answer: They record who performed sensitive actions and provide traceability for security and operational review.

119. How do you secure destructive actions?

Model answer: Destroy/cleanup paths require explicit admission/confirmation and are separated from normal deployment actions.

120. Why is least privilege difficult in a PaaS?

Model answer: The platform performs many conditional cloud actions across provisioning, release and cleanup, so permissions must reflect lifecycle capabilities rather than a single API call.

121. How does capability verification help least privilege?

Model answer: It makes required AWS actions explicit and testable per lifecycle path instead of relying on undocumented permission assumptions.

122. What is the benefit of OIDC credential lifetime?

Model answer: Credentials are short-lived and issued for the workflow identity, reducing exposure compared with reusable static keys.

123. How do you handle unsupported or suspicious repository structures?

Model answer: The admission layer rejects ambiguous, unsafe or unsupported build/service boundaries rather than executing arbitrary paths.

124. Does containerization make the app fully secure?

Model answer: No. Containers improve packaging and isolation but still require vulnerability management, least privilege, network controls and secure runtime configuration.

125. What future security improvements would you add?

Model answer: SBOMs, signed provenance, image signature verification, stronger policy-as-code and automated dependency/supply-chain attestations.

Observability, AI, FinOps and Product

126. What observability does DeployGuard provide?

Model answer: Pipeline stage metrics, GitHub workflow duration, AWS runtime metrics, CloudWatch logs, Prometheus export, Grafana dashboards and developer-facing health projections.

127. Why distinguish monitoring from troubleshooting?

Model answer: Monitoring shows state and telemetry; troubleshooting correlates evidence to a specific failure and recommends action.

128. How does AI troubleshooting start?

Model answer: A user opens a failed eligible pipeline run or verified live-runtime issue with sanitized evidence and starts an analysis session.

129. What evidence can the AI use?

Model answer: Persisted pipeline facts, structured failure diagnostics, bounded workflow/cloud/runtime evidence and conversation context.

130. Is AI allowed to decide the authoritative root cause?

Model answer: No. Deterministic diagnostic contracts and evidence remain authoritative; AI explains and recommends within those boundaries.

131. What happens if the AI provider is down?

Model answer: The subsystem falls back to evidence-only/local diagnostic output so the user still receives structured guidance.

132. How are AI sessions audited?

Model answer: Start, regenerate, follow-up, fallback and close actions are persisted/audited with project and pipeline context.

133. What is FinOps in DeployGuard?

Model answer: Pre-deployment cost estimation, resource breakdown, thresholds, tier limits, approvals and persisted cost evidence.

134. Why estimate cost before deployment?

Model answer: Cloud resources can be technically valid but economically unsuitable; cost is another deployment admission dimension.

135. What happens when a cost requires approval?

Model answer: The pipeline can pause until an authorized user approves the estimate, then resume.

136. What billing features exist?

Model answer: Plans, subscriptions, checkout, invoices, payments, usage counters and provider webhook handling.

137. How is cost different from billing?

Model answer: Cost estimates infrastructure consumption; billing is the commercial relationship with the platform user.

138. How would you measure platform success?

Model answer: Deployment success rate, lead time, rollback/recovery time, change failure rate, cost accuracy, developer effort and support/troubleshooting time.

139. What is the biggest current scalability risk?

Model answer: Per-project cloud-resource overhead and cost, especially dedicated routing/infrastructure for many low-traffic applications.

140. What is your strongest FYP-II demo sequence?

Model answer: Connect repository -> deploy -> show workflow and AWS evidence -> open LIVE app -> show monitoring/release -> destroy cleanly.

Rapid-Fire Technical Facts

Topic	Answer
Frontend	React 18 + Vite
Backend	NestJS 11 + TypeScript + TypeORM
Control-plane DB	PostgreSQL
Queue	BullMQ + Redis
IaC	Terraform
Primary cloud	AWS
Runtime	ECS/Fargate
Registry	ECR
Ingress	Application Load Balancer
Persistent database storage	EFS
Secrets	AWS Secrets Manager
Managed databases	PostgreSQL, MySQL, MongoDB
Monitoring	CloudWatch + Prometheus + Grafana
Security scan	Trivy
Source identity	Exact Git commit SHA
Build contracts	JS standalone, JS workspace member, Python standalone
Rollback	Exact historical immutable release; no rebuild
Destroy authority	Terraform plus deletion verification
AI troubleshooting	Sanitized evidence-first, advisory, provider fallback

Cost	Plan-derived estimate + policy/approval

30-Second Opening Defense

DeployGuard is a PaaS-style deployment control plane. A developer connects a GitHub repository, and DeployGuard resolves the exact source, service, build and runtime contract; creates and validates an immutable container image; provisions AWS infrastructure through Terraform; deploys to ECS; verifies runtime and public health; and only then records a stable release. Our FYP-II contribution is the deterministic lifecycle around those tools - including security, managed databases, release generations, rollback, destroy, state reconciliation, observability, cost governance and evidence-driven troubleshooting.

Three Statements to Avoid

- Avoid saying “we built AWS/Terraform/GitHub Actions.” Say that DeployGuard orchestrates these dependencies through its own control-plane contracts.
- Avoid saying “we support every web application.” State the current bounded JavaScript/Python deployment contract.
- Avoid claiming a live PASS without showing the exact run/release evidence. Separate implemented invariants from externally executed proof.